ROME Demo Assignment

Jiangyue Chu   jc13605

Part I

```
hier_gen(submodules)
```

```
Error compiling testbench
Testbench ran successfully
Total time:  379.5307946205139
Generation time:  53.59698224067688
Error handling time:  325.93381094932556
Error compiling testbench
Testbench ran successfully
Total time:  419.450227022171
Generation time:  67.96003675460815
Error handling time:  351.49018931388855
Error compiling testbench
Error compiling testbench
Error compiling testbench
Testbench ran successfully
Total time:  464.796190738678
Generation time:  158.0982894897461
Error handling time:  306.69789958000183
Overall Total time:  1263.777212381363
Overall Generation Time:  279.6553084850311
Overall Error handling time:  984.1218998432159
```

```
!vvp mux2to1/mux2to1
!vvp mux4to1/mux4to1
!vvp mux8to1/mux8to1
```

```
passed!
passed!
passed!
```

First I tried to use gpt-5 to run, although it runs successfully, it took 984 seconds, so I decided to use gpy-4o to run again, and run the rest of the assignment parts.

```
hier_gen(submodules)
```

```
Error compiling testbench
Error running testbench
Error compiling testbench
Testbench ran successfully
Total time:  14.73368787765503
Generation time:  3.2390635013580322
Error handling time:  11.494621992111206
Testbench ran successfully
Total time:  4.08673620223999
Generation time:  1.3256149291992188
Error handling time:  2.761120557785034
Error compiling testbench
Error compiling testbench
Error compiling testbench
Testbench ran successfully
Total time:  19.131274700164795
Generation time:  7.397238731384277
Error handling time:  11.734032392501831
Overall Total time:  37.951698780059814
Overall Generation Time:  11.961917161941528
Overall Error handling time:  25.98977494239807
```

```
!vvp mux2to1/mux2to1
!vvp mux4to1/mux4to1
!vvp mux8to1/mux8to1
```

```
passed!
passed!
passed!
```

The screen shot shows he feedback loop automatically corrected compilation errors. Final modules passed simulation.

Part II

```
hier_gen(submodules_part2, max_iterations=15)
```

```
Error running testbench
Testbench ran successfully
Total time:  6.113650798797607
Generation time:  2.077577829360962
Error handling time:  4.036072492599487
Testbench ran successfully
Total time:  4.866234302520752
Generation time:  1.356428861618042
Error handling time:  3.5098049640655518
Warnings compiling testbench
Testbench ran successfully
Total time:  11.10983395576477
Generation time:  4.713640213012695
Error handling time:  6.396192789077759
Error compiling testbench
Warnings compiling testbench
Error compiling testbench
Error compiling testbench
Error compiling testbench
Error compiling testbench
Error compiling testbench
Testbench ran successfully
Total time:  63.65125012397766
Generation time:  32.85202431678772
Error handling time:  30.799220085144043
Overall Total time:  85.74096918106079
Overall Generation Time:  40.99967122077942
Overall Error handling time:  44.74129033088684
```

```
!vvp half_adder/half_adder
!vvp full_adder/full_adder
!vvp adder4/adder4
!vvp adder8/adder8
```

```
passed!
passed!
passed!
passed!
```

Part III

Iteration 0

log_iter_0.txt ✕ log_iter_1.txt log_iter_2.txt log_iter_7.txt

```
{'role': 'system', 'content': 'You are an autocomplete engine for Verilog co
{'role': 'user', 'content': '//We are generating a 8-bit ripple-carry adder
{'role': 'assistant', 'content': '```verilog\nmodule half_adder(input wire a

 Iteration status: Error compiling testbench

```

Iteration 1

log_iter_0.txt log_iter_1.txt ✕ log_iter_2.txt log_i

```
NE Verilog module. CRITICAL FORMATTING: Return ONLY a sin
, full adder,  defined as follows:module half_adder(input
  // Logic for sum and carry\n    assign sum = a ^ b;
8tb.v:8: error: Unknown module type: adder8\n2 error(s) d
  // Logic for sum and carry\n    assign sum = a ^ b;



```

error: Unknown module type: adder8\n2 error(s) during elaboration.

Iteration 2

:t log_iter_2.txt ✕ log_iter_7.txt

```
': 'You are an autocomplete engine for Verilog code. Given a Verilog module specification, output ONLY ONE Verilog module. CRITICAL FORMATTING: Return ONLY a single f
 '//We are generating a 8-bit ripple-carry adder hierarchically in Verilog. We have generated half adder, full adder,  defined as follows:module half_adder(input wire
ent': '```verilog\nmodule half_adder(input wire a, input wire b, output wire sum, output wire carry);\n    // Logic for sum and carry\n    assign sum = a ^ b;    // X
 'The testbench compiled with warnings. Please fix the module. The output of iverilog is as follows:\n./adder8tb.v:11: warning: Port 4 (sum) of adder8 expects 8 bits,
ent': '```verilog\nmodule half_adder(input wire a, input wire b, output wire sum, output wire carry);\n    // Logic for sum and carry\n    assign sum = a ^ b;    // X
```

The testbench compiled with warnings. Please fix the module. The output of iverilog is as follows:\n./adder8tb.v:11: warning: Port 4 (sum) of adder8 expects 8 bits

In iteration 2, I didn't find the error. It might because the loop previously observed a width-mismatch warning in the testbench connection to sum (expected 8 bits, got 9). I furthered checked iteration 3.

Iteration 3

log_iter_3.txt

```
erilog module. CRITICAL FORMATTING: Return ONLY a single fenced code blc
ll adder,  defined as follows:module half_adder(input wire a, input wire
/ Logic for sum and carry\n    assign sum = a ^ b;    // XOR for sum\n
v:9: error: port ``A'' is not a port of DUT.\n./adder8tb.v:9: error: por
/ Logic for sum and carry\n    assign sum = a ^ b;    // XOR for sum\n



```

error: port ``A'' is not a port of DUT.

Iteration 7

log_iter_0.txt log_iter_1.txt log_iter_2.txt log_iter_7.txt ×

~~~
{'role': 'system', 'content': 'You are an autocomplete engine for Verilog code. Given a Verilog module specif
{'role': 'user', 'content': '//We are generating a 8-bit ripple-carry adder hierarchically in Verilog. We hav
{'role': 'assistant', 'content': '```verilog\nmodule half_adder(input wire a, input wire b, output wire sum,
{'role': 'user', 'content': "The testbench failed to compile. Please fix the module. The output of iverilog i
{'role': 'assistant', 'content': '```verilog\nmodule half_adder(input wire a, input wire b, output wire sum,

Iteration status: Testbench ran successfully

~~~